Reverse-Engineering with Ghidra

(Know your enemy!)

– TD (8) –

Abstract

The goal of these exercises is to get you acquainted with the Ghidra reverse-engineering tool. You will learn how to use it to analyze and understand the behavior of compiled programs, both in 32-bit and 64-bit architectures. The exercises will cover basic usage of Ghidra and a tour of its main tools.

1 Starting with Ghidra

Questions

1. **Installation and Setup.** Get the latest version of Ghidra from the official website (https://ghidra-sre.org/) and install it on your account.

2. **Creating a New Project.** Get the code material for this labwork from the course website. Then, open Ghidra and create a new project. Import the binary executable called `exercise-1-32` into your project. Start analyzing the binary and explore its structure. Do not forget to read the source code of the program to better understand the assembly and decompiled code.

```c
#include <stdlib.h>
#include <stdio.h>

int fibonacci(int n) {
  if (n <= 0) return 0;
  if (n == 1) return 1;
  return fibonacci(n - 1) + fibonacci(n - 2);
}

void display_fibonacci(int n) {
  printf("Fibonacci sequence up to %d: ", n);
  for (int i = 1; i <= n; i++)
    printf("%d ", fibonacci(i));
  printf("\n");
}

int main() {
  char buffer[16];
  printf("Enter your name: ");
  fgets(buffer, sizeof(buffer), stdin);
  printf("Hello, %s!\n", buffer);

  int num;
  puts("Enter a number: ");
  scanf("%d", &num);
  if (num < 0) {
    printf("Negative numbers are not allowed.\n");
    return EXIT_FAILURE;
  }
  display_fibonacci(num);

  return EXIT_SUCCESS;
}
```

3. **Symbol Tree.** Locate the `main()`, `display_fibonacci()`, and `fibonacci()` functions in the symbol tree. Explore the disassembled code and the decompiled code for each of these functions. And, if needed, try to annotate the code or to fix errors of the decompiler. Try to read the C code in parallel with the assembly and decompiled code to understand how the original C code was translated into assembly and then decompiled.

4. **Control Flow Graph.** For the `display_fibonacci()` function, explore the control flow graph (CFG) in Ghidra. First, select the function in the **Symbol Tree** and then open the CFG view by clicking on the toolbar button called "*Display Function Graph*". The result should be something close to Fig. 1.

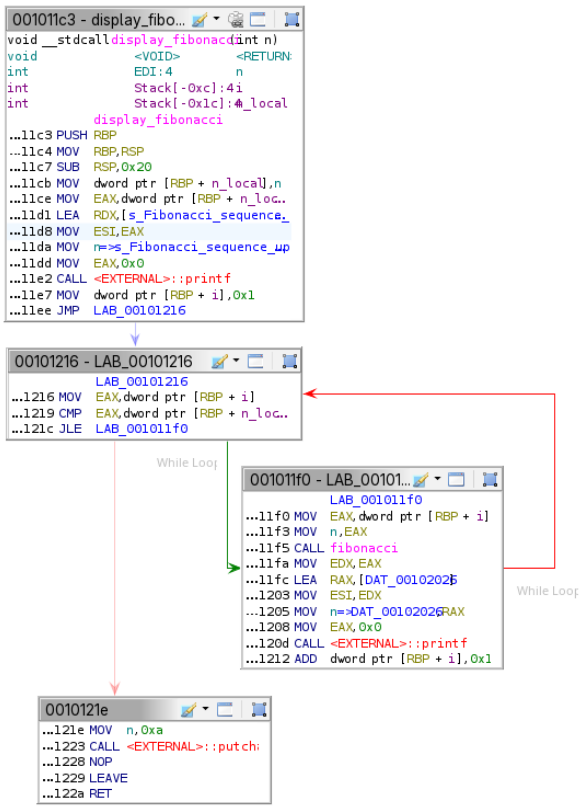

Figure 1: Control Flow Graph of the `display_fibonacci()` function.

5. **Function Call Graph.** Go to the `Windows` menu and open the **Function Call Graph** view when on the `main` function. Then, unfold the `display_fibonacci()` function to see the calls made by this function.

6. **Explore amd64 Assembly.** Import the `exercise-1-64` binary into your project and explore its assembly and decompiled code. Compare it with the 32-bit version and try to understand the differences between the two architectures.

7. **Start Annotating.** Start annotating the code of the `exercise-1-64` binary to better understand it. You can start by renaming variables and functions to more meaningful names and adding comments to explain the code when the decompiler did not managed to find the original name.

8. **Understand an Unknown Function.** In the `exercise-1-32` and `exercise-1-64` binaries, there is a call to an unknown function called `FUN_<address of function>`. Try to understand what this function does by analyzing its code and the way it is called.

9. **Stripped Binaries.** The `exercise-1-32` and `exercise-1-64` binaries are not stripped, which means that they contain the symbol information. This is not always the case in real-world scenarios. Strip the symbols from these binaries and then import the stripped versions into your project.

```
strip -o exercise-1-32-stripped exercise-1-32
strip -o exercise-1-64-stripped exercise-1-64
```

Select one of the stripped binaries and annotate it as much as you can with the help of the original source code. You should notice that the decompiled code is not as good as the one of the non-stripped version. Try to understand why this is the case.

2 Optimized and Stripped Binaries

Lets take a look to more complex binaries. For that we will take a bit more complex C program, compile it with full optimization and strip the symbols from the resulting binary. Look at the `Makefile` to see how the binaries are compiled and stripped.

As you should have noticed in the previous example, stripping a binary means to remove the symbol information from it. This is often done to reduce the size of the binary. But, another side effect of stripping is that it makes the reverse-engineering process more difficult, as the names of functions, variables and debug information are no longer available.

```c
#include <stdlib.h>
#include <stdio.h>

#include <string.h>

int g_multiplier = 2;

typedef struct {
    int id;
    char label[4];
    double score;
} result_t;

result_t process_data(int id, const char* name, double base) {
    result_t r;
    r.id = id;

    strncpy(r.label, name, sizeof(r.label) - 1);
    r.label[sizeof(r.label) - 1] = '\0';

    r.score = (base * (double)g_multiplier);

    return r;
}

int main(int argc, char** argv) {
    char name[4] = {0};

    if (argc < 2) {
        printf("Usage: %s <name>\n", argv[0]);
        return EXIT_FAILURE;
    }

    strncpy(name, argv[1], sizeof(name) - 1);
    name[sizeof(name) - 1] = '\0';

    result_t result = process_data(101, argv[1], 4.5);

    for (int i = 0; i < g_multiplier; i++) {
        printf("Result [%d]: %s (Score: %.2f)\n",
                result.id, result.label, result.score);
    }
```

```
    return EXIT_SUCCESS;
}
```

Questions

1. **Rename the Functions.** Start by importing the `exercise-2-32` binary into your project and explore its assembly and decompiled code. Then, try to rename all the functions back to their original names with the help of the source code.

2. **Rename the Variables.** Do the renaming process for as most of the variables as you can. You should notice that some variables seems to have disappeared and others seems to have popped from nowhere. Try to understand why this is the case by looking at the assembly code and the decompiled code.

3. **Fix Variables Types.** You must have noticed that some variables have the wrong type in the decompiled code. Usually pointers have lost the type on what they point to, but arrays can also be affected (structures also but we will see this later on). Try to fix the types of as many variables as you can (with the Retype feature). Use the source code to help you in this task.

4. **Create Custom Data Types.** In the source code, there is a structure called `result_t`. This structure is not properly identified in the decompiled code. Create a custom data type with the **Data Type Manager** and use it to fix the type of the variables that are supposed to be of this type.

   1. First, import the structure definition through **File** → **Parse C Source**.
   2. Once the **Parse C Source** is open, click on the '+' icon of the 'Source files to parse' section and select the `exercise-2.c` source file. Then, click on the '**Parse**' button to start the parsing process.
   3. After the parsing is done, check that the `result_t` structure is now available in the **Data Type Manager** (use the search bar to find it).
   4. Then, use the `result_t` structure to fix the type of the variables that are supposed to be of this type in the decompiled code and observe the changes on the decompiled code.
   5. Finally, rerun the decompilation to update the decompiled code with the new types and check that the code is now properly decompiled.

5. **Fix Function Signatures.** The function signatures are also affected by the stripping process. Try to fix the signatures of as many functions as you can (with the Edit Function Signature feature). Use the source code to help you in this task.

   Note that, in C, when a function is returning a structure, the caller is responsible to allocate the memory for the structure and to pass a pointer to this memory as the first argument of the function. This is why the `process_data()` function has a void return type in the original C code but a pointer return type in the decompiled code. Try to fix this by changing the return type of the function to `result_t` and by removing the first argument of the function.

6. **Creating Custom Types without the Source Code.** Now, take the `exercise-2-64` binary and and repeat the same process as before but without using the source code to get the original structure definition.

   Go to the **Data Type Manager** and right-click on the `exercise_2_64` data types node and select **New** → **Structure...**. Then, the **Structure Editor** will open and you can start creating the structure by adding fields to it.

3 BSim: Registering Functions and Variables

BSim is a way to transfert the information about functions and variables from one binary to another. This is useful when you are analyzing a family of binaries that are similar but not identical, for example different versions of the same software. By registering the functions and variables of one binary, you can then apply this information to other binaries of the same family to speed up the reverse-engineering process.

Questions

1. **Registering Functions and Variables.** Take the `exercise-2-32` binary and register all the functions and variables that you have identified in the previous exercises.

    1. **Create a BSim database:** Go to the **ScriptManager** and run the `CreateH2BSimDatabase.java` script to create a BSim database for the `exercise-2-32` binary.
    2. **Connect BSim to the database:** Go to **BSim** → **Manage Servers**, then create a new server with the '+' icon and select the **File** option. Connect the server to the file your create just before.
    3. **Populate your Database with your findings:** Go to the **ScriptManager** and execute the script `AddProgramToH2BSimDatabaseScript.java`

2. **Applying the BSim Database.** Now, load the `exercise-3-32` binary into the code browser. Once in the code browser, select all the binary code (Ctrl+A) and go to **BSim** → **Search Functions...**, select the server that you created in the previous question. Then, click on the '**Search**' button.

    After the search is done, a list of matches will be displayed. Select the one matching `process_data()` and right-click on it, select **Apply Name** and **Apply Signature and Data Types**.

    You should see that the `process_data()` function is now properly named and typed in the decompiled code. You can also check that the variables of this function are now properly typed as well.

4 Reversing Hero

Reversing Hero is a challenge that consists in reversing a binary to find a secret flag. The binary is a simple CTF-style program that takes a string as input and checks if it is the correct flag. The goal of this challenge is to apply the skills learned in the previous exercises to reverse-engineer the binary and find the flag.

Go get the binary on the course website or on `www.reversinghero.com` and try to reverse it with Ghidra. You can also use other tools if you want to, but Ghidra should be enough to solve the 15 levels of the challenge.

Questions

1. **Level 1.**
2. **Level 2.**
3. **Level 3.**
4. **Level 4.**
5. **Level 5.**
6. **Level 6.**
7. **Level 7.**
8. **Level 8.**
9. **Level 9.**
10. **Level 10.**
11. **Level 11.**
12. **Level 12.**
13. **Level 13.**
14. **Level 14.**
15. **Level 15.**